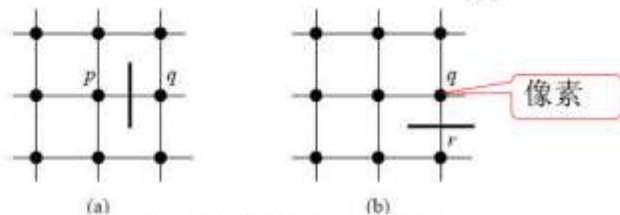




6.1.1 图搜索

- 对任一段弧 (n_i, n_j) 都可定义一个代价，记为 $c(n_i, n_j)$
- 通路 $n_1, n_2, n_3, \dots, n_k$ 的总代价为 $C = \sum_{i=2}^k c(n_{i-1}, n_i)$



$$c(p, q) = H - [f(p) - f(q)]$$

- 其中H为图像中最大灰度值， $f(p)$ 代表p点的灰度值

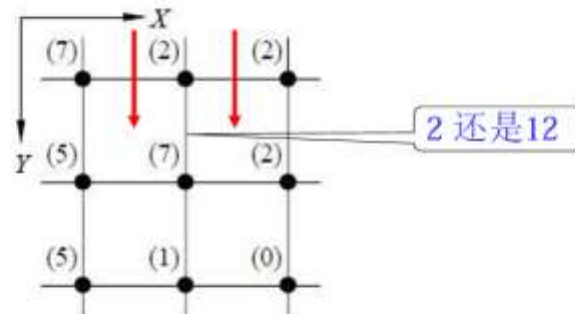
10



6.1.1 图搜索

$$c(p, q) = H - [f(p) - f(q)]$$

- 思考：轮廓搜索中图结构是有向图还是无向图？

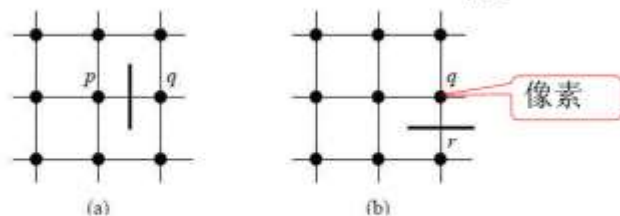


12



6.1.1 图搜索

- 对任一段弧 (n_i, n_j) 都可定义一个代价，记为 $c(n_i, n_j)$
- 通路 $n_1, n_2, n_3, \dots, n_k$ 的总代价为 $C = \sum_{i=2}^k c(n_{i-1}, n_i)$



$$c(p, q) = H - [f(p) - f(q)]$$

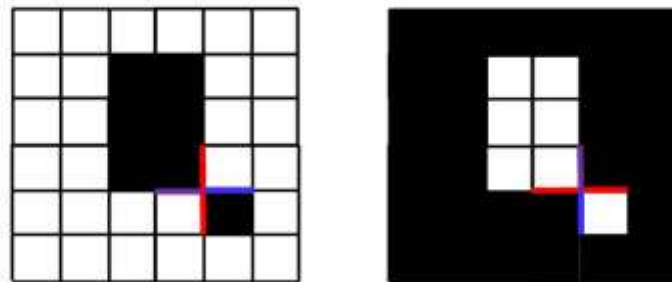
- 与灰度差成反比
- 与梯度大小成反比



6.1.1 图搜索

$$c(p, q) = H - [f(p) - f(q)]$$

- 思考：轮廓搜索中图结构是有向图还是无向图？
- 定义：顺着边缘方向，p为右边点，q为左边点



如果把它变成无向图会怎样？

意味着你必须把代价变成绝对值（比如上下走代价都是 2）。这样一来，算法只知道“这里有一条强烈的边界”，但它**彻底失去了方向感**。它不知道这条边界的左边是暗的还是右边是暗的。当图像中靠得很近的地方有多个不同物体时，算法就极易在不同物体的边界上“反复横跳”，根本无法紧紧贴合你真正想要的那个目标的外轮廓。

2. 算法会频繁“回头”（产生死循环或打结）

我们在做轮廓搜索时，底层用的是图搜索算法（比如 Dijkstra 或 A*，课件后面也有提到）。这些算法的核心逻辑就是“永远找当前代价最小的路往下走”。

如果这是一个无向图，意味着“往前走一步”和“退回上一步”的代价是一模一样的（通常都很小，比如是 2）。

当算法沿着边缘走到一个稍微有些模糊或复杂的区域时，它会发现：哎？退回去的代价也是 2，也是最小代价。这时候算法就很容易在一条边上来回打转、形成死循环，或者最后提取出的轮廓线条像乱麻一样打结在一起。

有向图完美解决了这个问题：

一旦你顺着正确的方向（比如“右亮左暗”，代价为 2）往前走，如果你想回头，代价瞬间飙升到 12。这就相当于在边界上强行画了“单行道”的标志，逼着搜索算法只能勇往直前地把轮廓闭合，绝不允许回头。

3. 违背了“闭合轮廓”的物理意义

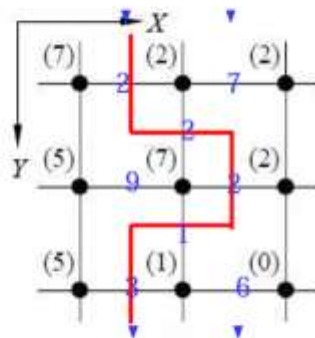
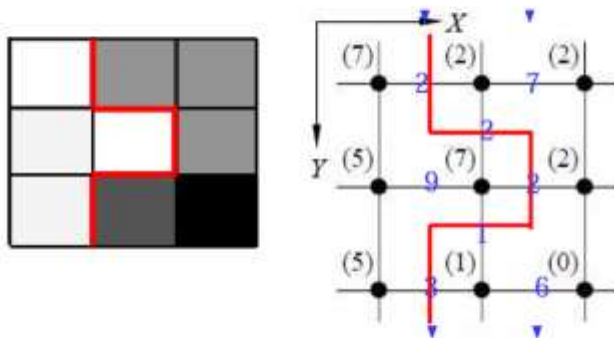
图像分割的最终目的，通常是找出一个**封闭**的连通区域。一个封闭的几何图形轮廓，必然是有明确的“内”和“外”之分的。

有向图通过规定“右边是 p ，左边是 q ”，实际上是在用算法模拟我们物理世界中“用手摸着物体的外边缘绕一圈”的动作。无向图是毫无内外概念的线条集合，只有有向图，才能严谨地圈出一个真正的“目标物体”。



6.1.1 图搜索

- 代价函数 $c(p, q) = H - [f(p) - f(q)]$
- 利用图搜索技术从上向下可检测出如图所示的对应大梯度边缘元素的边界段



14



6.1.1 图搜索

$c(p, q) = H - [f(p) - f(q)]$
定义：顺着边缘方向，
p为右边点，q为左边点

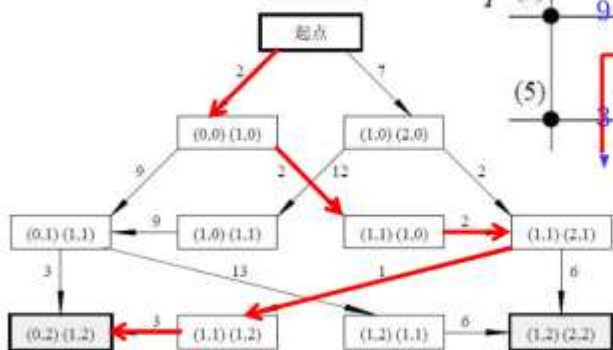
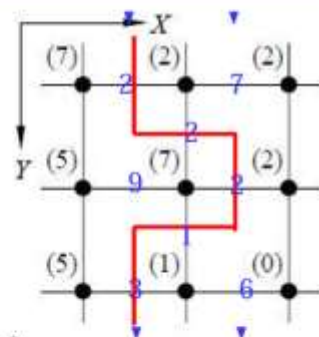


图 6.1.3 用于检测边界的搜索图



15



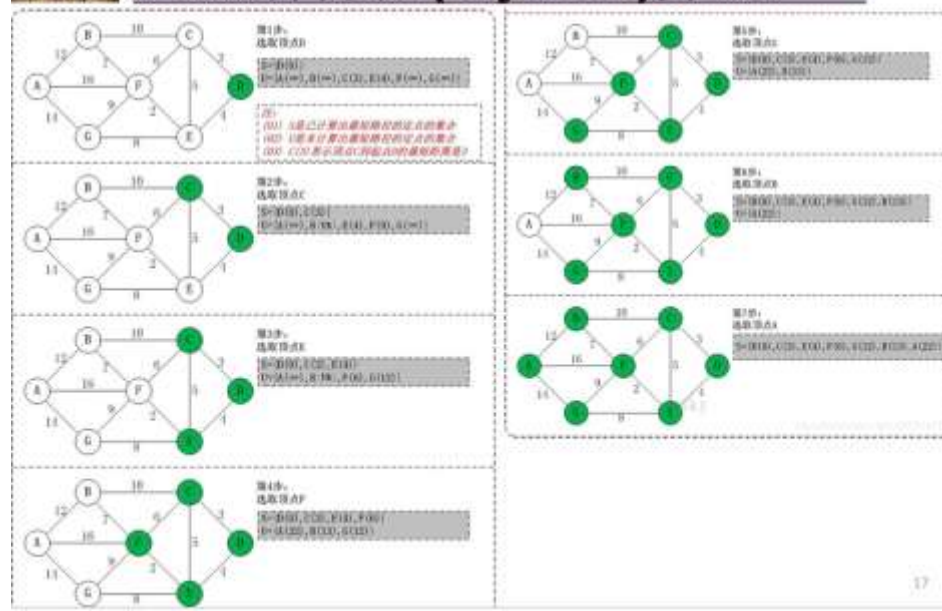
迪杰斯特拉(Dijkstra)算法

- 通过Dijkstra计算图G中的最短路径时，需要指定一个起点D(即从顶点D开始计算)。
- 此外，引进两个数组S和U。S的作用是记录已求出最短路径的顶点(以及相应的最短路径长度)，而U则是记录还未求出最短路径的顶点(以及该顶点到起点D的距离)。
- 初始时，数组S中只有起点D；数组U中是除起点D之外的顶点，并且数组U中记录各顶点到起点D的距离。如果顶点与起点D不相邻，距离为无穷大。
- 然后，从数组U中找出路径最短的顶点K，并将其加入到数组S中；同时，从数组U中移除顶点K。接着，更新数组U中的各顶点到起点D的距离。
- 重复第4步操作，直到遍历完所有顶点。

16



迪杰斯特拉(Dijkstra)算法



17



6.1.2 动态规划 (A*算法)

- 借助有关具体问题的启发性知识减少搜索
- 图搜索的算法由以下几个步骤构成
 - (1) 将起始结点标记为OPEN并置 $g(s) = 0$
 - (2) 如果没有结点OPEN, 失败退出, 否则继续
 - (3) 将根据式 $r(n) = g(n) + h(n)$ 算得的估计代价 $r(n)$ 为最小的OPEN结点标记为CLOSE
 - (4) 如果 n 是目标结点, 找到通路 (可由 n 借助指针上溯至 s) 退出, 否则继续

21



6.1.2 动态规划

- (5) 展开结点 n , 得到它的所有子结点 (如果没有子结点, 返回步骤(2))
- (6) 如果某个子结点 n_i 还没有标记, 置 $g(n_i) = g(n) + c(n, n_i)$, 标记它为OPEN并将指向它的指针返回到结点 n
- (7) 如果子结点 n_i 已标记为OPEN或CLOSE, 根据 $g'(n_i) = \min[g(n_i), g(n) + c(n, n_i)]$ 更新它的值。将其 g' 值减小的CLOSE子结点标记为OPEN, 并将原来指向所有其 g' 值减小的子结点的指针重新指向 n 。返回步骤(2)

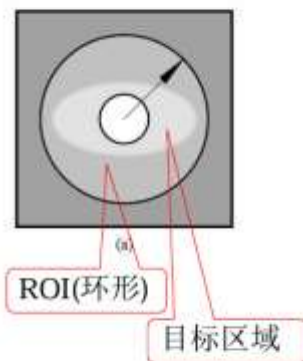


6.1 轮廓搜索

- 与边缘检测技术比较：
 - 边缘检测等分割属于并行边界技术
 - 图搜索、动态规划属于串行边界技术
 - 轮廓搜索实际上将边缘点的检测融入代价函数的计算，从而把边缘检测和边界连接结合起来

封闭轮廓搜索

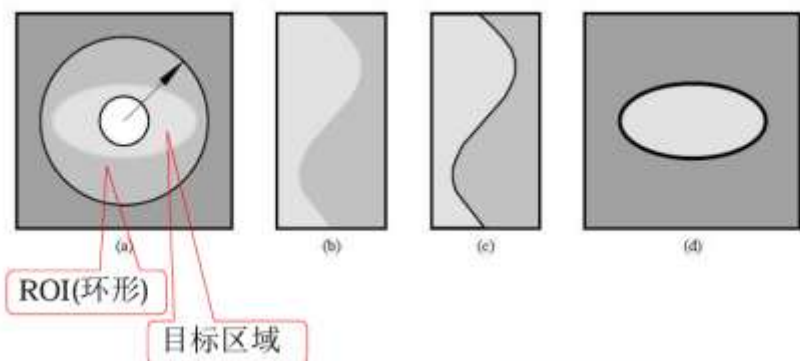
- 对图像进行极坐标变换
- 同时解决确定起始点和判断搜索是否结束这两个问题



26

封闭轮廓搜索

- 对图像进行极坐标变换
- 同时解决确定起始点和判断搜索是否结束这两个问题



27

6.2 主动轮廓模型

- 6.2.1 主动轮廓
- 6.2.2 能量函数

- 主动轮廓是一种基于边缘信息的目标分割方法
- 思想：初始封闭曲线演化逼近目标轮廓
- 主动轮廓模型也称蛇模型

29

6.2 主动轮廓模型

- 基本原理是表征拟合误差的“能量”为最小化的曲线。
- 设对于拟合目标有一个待选曲线集,定义能量函数与待选集中每一条曲线相关联,能量函数的设计原则就是:有利属性要能导致能量缩小。
- 有利属性包括:曲线连续、平滑、曲线与高梯度区域接近以及其他一些具体的先验知识。
- 活动轮廓在取值范围内移动时,就能在能量函数的指导下收敛到局部边界,且能保持曲线的连续和平滑。

30



6.2 主动轮廓模型

图像上一组排序的点的集合

$$V = \{v_1, \dots, v_L\}$$

$$v_i = (x_i, y_i), i = \{1, \dots, L\}$$

$$E_i(v_i') = \alpha E_{\text{int}}(v_i') + \beta E_{\text{ext}}(v_i')$$

内部能量函数

外部能量函数

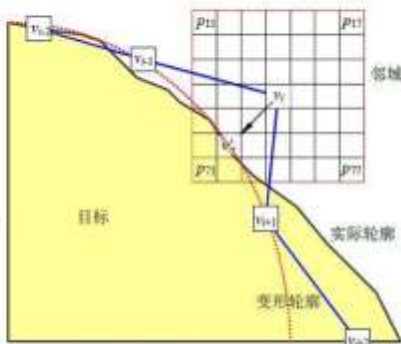


图 6.2.1 主动轮廓上点的移动

32



6.2 主动轮廓模型

1. 内部能量函数

$$E_{\text{int}} = 1/2 (\alpha |v'(s)|^2 + \beta |v''(s)|^2)$$

(1) 弹性势能

(2) 弯曲势能

$v'(s)$ 和 $v''(s)$ 是 v 对 s 的一阶和二阶导数, 系数 α 、 β 分别是控制蛇模型的弹性和刚性, 这些参数操纵着模型的物理行为和局部连续性

1. 一阶导数 $v'(s)$ 与弹性势能 (Elasticity)

- **数学本质:** 在参数化曲线中, $|v'(s)|$ 代表了曲线的一阶变化率。如果我们将 s 视为单位步长, 那么 $|v'(s)|$ 衡量的是相邻两点之间的距离 (步长大小)。
- **物理类比:** 这项能量正比于距离的平方, 极像物理学中的胡克定律 ($E = 1/2 kx^2$)。它模拟了一根橡皮筋的张力。
- **作用效果:**
 - **防止断裂/拉伸:** 当相邻点距离变大时, 一阶导数值增加, 能量上升。为了最小化能量, 算法会迫使点与点之间靠得更近。
 - **长度收缩:** 这项能量的存在会让轮廓产生整体收缩的趋势, 倾向于缩短总周长。因此, 系数 α 控制着这种“橡皮筋”的强度 (弹性)。

2. 二阶导数 $v''(s)$ 与弯曲势能 / 刚性 (Stiffness)

- **数学本质:** $v''(s)$ 代表曲线的二阶变化率, 在数学上它与曲率 (Curvature) 成正比。它衡量的是曲线方向改变的速度。
- **物理类比:** 这项能量模拟了一根有韧性的细铁丝在被弯曲时产生的内应力。
- **作用效果:**
 - **惩罚尖角:** 如果轮廓在某个地方突然转折 (形成尖角), 该处的二阶导数会变得极大, 导致能量飙升。
 - **保持平滑:** 为了最小化能量, 轮廓会尽量避免剧烈的方向变化, 使形状变得圆滑。
 - **抵抗变形:** 系数 β 越大, 这根“铁丝”就越硬 (刚性强), 轮廓就越难被迫做出复杂的弯曲动作, 更倾向于保持直线或圆滑的弧线。

总结

- α (弹性系数): 决定轮廓是否容易被拉长。它主要约束长度, 防止点分布太稀疏。
- β (刚性系数): 决定轮廓是否容易被折弯。它主要约束形状, 防止出现锯齿或尖锐转角。



6.2 主动轮廓模型

1. 内部能量函数

连续能量

$$E_{con}(v'_i) = \frac{1}{I(V)} ||v'_i - \gamma(v_{i-1} + v_{i+1})||^2$$

膨胀能量

$$E_{bal}(v'_i) = n_i \cdot [v_i - v'_i]$$



6.2 主动轮廓模型

2. 外部能量函数

$$\beta E_{\text{ext}}(v_i) = mE_{\text{mag}}(v_i) + gE_{\text{grad}}(v_i)$$

(1) 图像灰度能量

$$E_{\text{mag}}(v_i') = I(v_i')$$

将轮廓吸向高或低的灰度区域；

m为正，轮廓向低灰度区域移动

38



6.2 主动轮廓模型

2. 外部能量函数

$$\beta E_{\text{ext}}(v_i) = mE_{\text{mag}}(v_i) + gE_{\text{grad}}(v_i)$$

(2) 图像梯度能量

$$E_{\text{grad}}(v_i') = -\mathbf{n}_i \cdot \nabla I(v_i')$$

图像中梯度

将变形轮廓向图像边缘吸引

目标边界处法向量应和梯度方向接近

40

1. 第一步：计算切向量 (Tangent Vector)

为了求得点 v_i 处的法线，我们首先要找到这里的切线方向。

最常用且最稳定的方法是**中心差分 (Central Difference)**，即直接连接它的前驱点 v_{i-1} 和后继点 v_{i+1} 作为切线方向：

$$t_i = v_{i+1} - v_{i-1}$$

如果展开成坐标系中的 (x, y) ：

$$t_i = (x_{i+1} - x_{i-1}, y_{i+1} - y_{i-1})$$

(注：这里不用 v_i 与相邻点做差，是因为中心差分能更好地抵消局部坐标抖动，反映宏观走向。)

2. 第二步：旋转 90 度求法向量

在线性代数中，如果有一个二维向量 (x, y) ，将它逆时针或顺时针旋转 90 度，坐标会变成 $(-y, x)$ 或 $(y, -x)$ 。

因此，与切向量 t_i 垂直的未归一化法向量 N_i 可以直接写为：

$$N_i = -(y_{i+1} - y_{i-1}, x_{i+1} - x_{i-1})$$

或者

$$N_i = (y_{i+1} - y_{i-1}, -(x_{i+1} - x_{i-1}))$$

怎么决定选哪一个？（向外还是向内）

这取决于你的点集是**顺时针**还是**逆时针**排列的。

- 在“膨胀能量”的设定中，我们需要的是**向外的单位法向量**。
- 算法初始化时，会规定好点集的排列顺序，并固定选择能指向轮廓外部的那一种旋转方式。

3. 第三步：归一化 (Normalization)

因为公式里的 n_i 是**单位法向量**（只代表方向，不包含大小），我们需要除以它的模长：

$$n_i = \frac{N_i}{\|N_i\|}$$

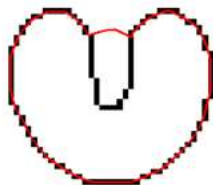


6.2 主动轮廓模型

- Snake模型的特点

- Snake模型的缺点

- 对初始位置敏感, 需要依赖其他机制将Snake放置在感兴趣的图像特征附近;
- 它有可能收敛到局部极值点, 甚至发散.
- 对于边界上的凹点无法有效跟踪 (图像边缘捕获域问题。尺度空间中由粗到精地极小化能量可以极大地扩展捕获区域和降低计算复杂性)
- 不能适应拓扑结构变化



总结

- 主动轮廓模型 (snake) 回顾
- 内部能量 $E_{int} = 1/2 (\alpha | \underline{v}'(s) |^2 + \beta | \underline{v}''(s) |^2)$
- 外部能量
 - 外部能量 E_{ext} 决定着向某种固定的特征移动蛇模型, 吸引蛇模型到显著的图像特征。因为这些特征只能根据特定的问题而定义, 所以一般的外部能量函数不易确定。因此, E_{ext} 没有统一的数学表达式, 必须从问题本身的特性出发, 根据实际情况处理



作业

6.1 将左图中指出的所有边缘元素都标在右图所示的图像上，并计算最小代价通路的代价。

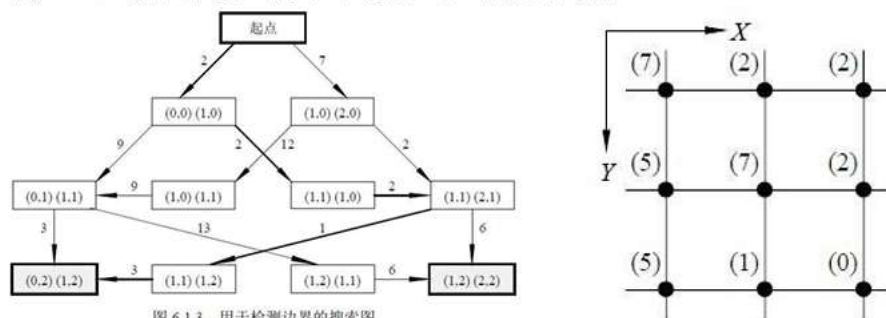


图 6.1.3 用于检测边界的搜索图

6.5 试讨论主动轮廓内部能量函数中加权系数 c 和 b 的作用，以及取值不同时（太大或者太小）对最终轮廓形状的影响

$$\alpha E_{\text{int}}(v_i) = cE_{\text{con}}(v_i) + bE_{\text{bal}}(v_i)$$